



Huy Dao <huydao@engineering.upenn.edu>

CIS 5980: feedback from our Phase 1 meeting (Project Beta)

Chris Callison-Burch <ccb@engineering.upenn.edu>

Tue, Sep 8, 2026 at 1:22 PM

To: huydao@engineering.upenn.edu, jauyeun@engineering.upenn.edu

Cc: moderr@seas.upenn.edu, dhanushg@seas.upenn.edu, kylems@seas.upenn.edu

Hi Henry and Jacky,

Thanks for a good conversation last night. As promised, here is the transcript of our part of the session (shared with you both and the TAs as a Google Doc): [CIS 5980 Phase 1 meeting transcript, 2026-09-07](#). Below is what we agreed, what I'd push on before the proposal is due on Sunday, September 13, and a bigger idea I want you to consider.

What we decided

The project is the explainer, not the trading bot. You are building a system that answers a trader's questions about decisions an automated system made, faithfully and only from the record, and refuses what the record can't support. Profitability is not the success criterion. I recommend that you stay in one trading category like equities; don't spend time on crypto or options.

Make the inspector the core, not the differentiator

Right now most of what's built is the quant pipeline. The AI component is the part furthest from done: your two-pager says the grounding of answers in the decision records isn't merged yet, and the "What we would cut" section makes the cross-artifact questions the first thing to go if it isn't merged by week 8. Flip that priority: shift your focus away from the training algorithms, since they aren't really modern AI, and instead focus your capstone on the inspector. Concretely, I'd expect a working inspector ASAP, not something that you delay until week 8.

Think of its job in three tiers, which also make a good evaluation set:

1. **Answerable from one decision record** (guaranteed core): "Why was this sized at 40 shares?", "Which breakout condition failed at 10:32?", "Was this rejection the filter's call or the risk engine's?", "What did each filter model score this signal?"
2. **Answerable, but needs several records** (your cross-artifact layer): "How often did the filter reject signals the raw rules would have taken this quarter, and how did those trades do?", "Which rule condition is the most common rejection reason in this fold?"
3. **Not answerable from the record**, where the correct behavior is a clean refusal: "Would this have worked with a tighter stop?", "Should I lower the threshold?", "Which strategy is best right now?"

Build ~200 questions across those tiers early. For tiers 1 and 2, decide whether each has a single checkable answer or a rubric; AutoRubric (<https://autorubric.org>, my PhD students' LLM-as-judge platform) is a good fit for rubric grading. For tier 3, the metric is the fraction of questions correctly refused. Add a contradiction rate: how often does an answer state something the record contradicts? Your 60-second audit test with three unfamiliar readers is a good product metric; keep it.

You also need a baseline. The simplest inspector is: hand Claude the strategy code, the data window, and the decision, and ask why. Measure that first. Your system has to beat it on faithfulness and refusal, and the gap is your story.

A bigger version worth considering

I recommend that you expand the scope of the project. Here is what I have in mind. Expand from “explain one strategy’s decisions” to a small simulated brokerage. Picture the AI version of a retail broker: user accounts with balances and positions, a support agent that answers questions about accounts and past trades, executes user-requested trades in a simulated market (“sell all my Microsoft”), and enforces policy (no investment advice, no speculation about hypotheticals). Your inspector becomes one capability of that agent. This gives you:

- **Use AI coding agents to build the simulated brokerage.** The brokerage itself (accounts, positions, a simulated market, an order book) is exactly the kind of well-specified software that Claude Code or Codex can build quickly from a clear spec, so it need not eat your schedule. Building and demoing a working simulated brokerage, and being able to talk about how you directed the coding agents to build it, would be a cool demo for potential employers.
- **Tool calling and MCP.** Once the simulated brokerage has an API, you can wrap it in MCP servers so that any agent, not just yours, can interact with it. This could include tools like: Account lookup, order placement, and historical prices become tools; the agent must check balances and holdings before acting.
- **Guardrails and red-teaming as a second evaluation track.** Can a user trade on someone else’s account? Can they talk the agent into advice? The June 2026 Meta incident, where attackers took over Instagram accounts (including Barack Obama’s) by simply asking Meta’s AI support agent to change the recovery email, is the cautionary tale; a brokerage agent with order-placement authority is exactly that risk. See Krebs on Security, June 2026: <https://krebsonsecurity.com/2026/06/hackers-used-metas-ai-support-bot-to-seize-instagram-accounts/>
- **Hallucination measured, then reduced.** “What was Apple trading at on September 7, 2008?” is a hallucination probe; adding a historical-data tool and showing before/after is a clean experiment.
- **A stretch goal with real AI depth.** Let a user describe a trading policy in English and have the agent generate and register the rule code, then explain that rule’s decisions later.

The result is both a product and a benchmark: a simulated environment other people could test their agents in.

Related work to read this week

- **τ -bench and τ^2 -bench (Sierra).** The closest analog to what I’m describing: a simulated user talks to an agent that has domain tools and a policy document, in retail and airline customer-service domains; tasks have one correct end state and the metric is pass^k reliability. Your brokerage would be a third domain in this mold. Paper: <https://arxiv.org/abs/2406.12045>; τ^2 paper: <https://arxiv.org/abs/2506.07982>; code: <https://github.com/sierra-research/tau2-bench>
- **WebArena.** Self-hosted, reproducible environments (shopping, forums, code hosting) for evaluating agents end to end; the model for building a standalone simulated environment with checkable outcomes. <https://arxiv.org/abs/2307.13854>
- **FinMCP-Bench.** Financial agents evaluated against real MCP tool servers, built from production financial-agent logs; relevant to your tool design. <https://arxiv.org/abs/2603.24943>
- **StockBench.** A benchmark for LLM agents trading stocks over multi-month windows. Read it to see what the “better trading bot” path looks like and why we’re not taking it. <https://arxiv.org/abs/2510.02209>

A possible third teammate

One more thing. Tolulope (Tolu) Akinlabi is looking to join another team; his team dissolved when a member dropped the class. He is on the AI Engineering track and has a background and interest in the economics of AI systems (token usage and inference cost), which would fit the cost and latency side of this project well. If a bigger scope appeals to you, a third pair of hands would help. Would you be interested in having him join? Let me or Derrick know and we can put you in touch.

For the proposal, due Sunday, September 13: the problem and audience you have are good; rewrite “what we are building” around the inspector (or the brokerage agent), and make faithfulness, refusal, and audit time the primary success metrics. Happy to look at a draft before you submit.

Best,
Chris